



UNIVERSIDAD PERUANA LOS ANDES



FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA
DE SISTEMAS Y COMPUTACIÓN

AUTOMATIZACIÓN Y MANTENIMIENTO

DE BASES DE DATOS CON

SQL SERVER



JOBS



ALERTS



MAINTENANCE
PLANS



POWERSHELL



MONITOREO
PROACTIVO



JOBS



ALERTS



MAINTENANCE
PLANS



POWERSHELL



CORREO

Base de Datos:

GRADOSTITULOS



SEGURIDAD



BACKUP



MONITOREO



AUTOMATIZACIÓN



ESTUDIANTE:

GUADALUPE CARBAJAL
EMANUEL GADDIEL



CARRERA:

INGENIERÍA DE SISTEMAS
Y COMPUTACIÓN



ASIGNATURA:

BASE DE DATOS II



DOCENTE:

MG. RAÚL FERNÁNDEZ BEJARANO



HUANCAYO - PERÚ

2026

TEMA 1. Creación de Jobs y Alerts con SQL Server Agent

Proyecto 1: Reporte Diario de Trámites Registrados

1) Enunciado

La Oficina de Grados y Títulos requiere que todos los días a las 08:00 AM se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior. El estudiante deberá crear un Job, un Step en T-SQL, programar el Job diariamente y registrar el resultado en una tabla de auditoría

2) Script de la solución

```
USE GradosTitulos;  
GO
```

-- 1. Creación de la tabla de Auditoría

```
IF OBJECT_ID('AuditoriaJobs', 'U') IS NULL  
BEGIN  
    CREATE TABLE AuditoriaJobs (  
        ID INT IDENTITY(1,1) PRIMARY KEY,  
        NombreJob VARCHAR(100) NOT NULL,  
        FechaEjecucion DATETIME DEFAULT GETDATE(),  
        Mensaje VARCHAR(MAX) NOT NULL  
    );  
END;  
GO
```

-- 2. Creación del Job, Step y Schedule (Se usa msdb)

```
USE msdb;  
GO
```

```
DECLARE @jobName VARCHAR(100) = N'Job_Reporte_Tramites_Diarios';  
DECLARE @jobId BINARY(16);
```

-- Eliminar el Job si ya existe para evitar duplicados

```
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @jobName)  
    EXEC msdb.dbo.sp_delete_job @job_name = @jobName;
```

-- Crear el Job

```
EXEC msdb.dbo.sp_add_job  
    @job_name = @jobName,  
    @enabled = 1,  
    @description = N'Reporte diario de trámites del día anterior para Grados y  
Títulos',  
    @job_id = @jobId OUTPUT;
```

-- Crear el Step del Job

```
EXEC msdb.dbo.sp_add_jobstep
```

```

@job_id = @jobId,
@step_name = N'Contar y Registrar Tramites',
@subsystem = N'TSQL',
@database_name = N'GradosTitulos',
@command = N'
DECLARE @Cantidad INT;
SELECT @Cantidad = COUNT(*)
FROM Tramites
WHERE CAST(FechaRegistro AS DATE) = CAST(DATEADD(day, -1, GETDATE())
AS DATE);

```

```

INSERT INTO AuditoriaJobs (NombreJob, Mensaje)
VALUES ("Job_Reporte_Tramites_Diarios", "Cantidad de trámites registrados el
día anterior: " + CAST(@Cantidad AS VARCHAR(10)));

```

```

,
```

```

@retry_attempts = 1,

```

```

@retry_interval = 5;

```

-- Crear la programación (Daily a las 00:00 AM)

```

EXEC msdb.dbo.sp_add_jobschedule

```

```

@job_id = @jobId,

```

```

@name = N'Schedule_Diario_GAM',

```

```

@freq_type = 4, -- Diaria

```

```

@freq_interval = 1,

```

```

@active_start_time = 000000 -- 00:00:00

```

-- Asociar el Job al servidor local

```

EXEC msdb.dbo.sp_add_jobserver @job_id = @jobId, @server_name = N'(local)';

```

```

GO

```

3) Resultado

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Explorador de objetos' (Object Explorer) shows the 'Trabajos' (Jobs) folder expanded, with 'Job_Reporte_Tramites_Diarios' selected. The central pane shows the 'SQLQuery1.sql' script, which includes the following code:

```

1 USE GradosTitulos;
2 GO
3
4 -- 1. Creación de la tabla de Auditoria
5 IF OBJECT_ID('AuditoriaJobs', 'U') IS NULL
6 BEGIN
7     CREATE TABLE AuditoriaJobs (
8         ID INT IDENTITY(1,1) PRIMARY KEY,
9         NombreJob VARCHAR(100) NOT NULL,
10        FechaEjecucion DATETIME DEFAULT GETDATE(),
11        Mensaje VARCHAR(MAX) NOT NULL
12    );
13 END;
14 GO
15
16 -- 2. Creación del Job, Step y Schedule (Se
17 USE msdb;
18 GO
19
20 DECLARE @JobName NVARCHAR(100) = N'Job_Reporte_Tramites_Diarios';
21 DECLARE @JobId BINARY(16);
22
23 -- Eliminar el Job si ya existe para evitar
24 IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE job_name = @JobName)
25 EXEC msdb.dbo.sp_delete_job @job_name = @JobName;

```

On the right, the 'Propiedades del trabajo - Job_Reporte_Tramites_Diarios' (Job Properties) window is open. The 'General' tab is selected, showing the following details:

- Nombre: Job_Reporte_Tramites_Diarios
- Propietario: EITR\Jeffe
- Categoría: [Sin categoría (Local)]
- Descripción: Reporte diario de trámites del día
- Conexión: Servidor: EITR; Conexión: EITR\Jeffe
- Habilitado: ☒
- Origen:
- Creado: 10/07/2026 20:09:57
- Última modificación: 10/07/2026 20:09:58
- Ejecutado por última vez:

At the bottom, the 'Mensajes' (Messages) pane shows a status message: 'SQLServerAgent is not currently running so it cannot be... No se encontraron problemas.' (No problems were found).

4) Justificación técnica de la solución aplicada

Se utiliza el subsistema de automatización SQL Server Agent mediante los procedimientos almacenados del sistema en la base de datos msdb (sp_add_job, sp_add_jobstep, etc.). El cálculo del "día anterior" se realiza dinámicamente restando un día a la función GETDATE() y convirtiendo ambos operandos a DATE para neutralizar el componente de la hora, asegurando que el conteo cubra la ventana exacta de 24 horas del día previo.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Se verifica la existencia previa de la tabla y del Job antes de su creación para evitar errores en ejecuciones consecutivas del script (Idempotencia).
- El Step incluye reintentos automatizados (@retry_attempts = 1) en caso de bloqueos transitorios.
- No se utiliza funciones escalares del lado de la columna de la tabla (FechaRegistro) para permitir un filtrado óptimo mediante el uso de índices SARGable.

Proyecto 2: Actualización Semanal de FechaUltMod

1) Enunciado Automatizar la actualización semanal del campo FechaUltMod para todos los trámites que permanezcan en estado EN PROCESO, registrando la ejecución en el historial del Job.

2) Script de la solución

USE msdb;

GO

DECLARE @JobName NVARCHAR(100) = N'Job_Actualizar_FechaMod_Semanales';

DECLARE @JobId BINARY(16);

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)

EXEC msdb.dbo.sp_delete_job @job_name = @JobName;

EXEC msdb.dbo.sp_add_job

@job_name = @JobName,

@enabled = 1,

@job_id = @JobId OUTPUT;

EXEC msdb.dbo.sp_add_jobstep

@job_id = @JobId,

@step_name = N'UPDATE FechaUltMod',

@subsystem = N'TSQL',

@database_name = N'GradosTítulos',

@command = N'

```

UPDATE Trámite

SET FechaUltMod = GETDATE()

WHERE Estado = "EN PROCESO";

PRINT "Actualización de trámites EN PROCESO completada exitosamente."

;

@retry_attempts = 2;

-- Programación Semanal Corregida (Cada 1 semana, los Domingos a las 00:00 AM)

EXEC msdb.dbo.sp_add_jobschedule

    @job_id = @JobId,

    @name = N'Schedule_Semanal_Domingo',

    @freq_type = 8,          -- 8 = Semanal

    @freq_interval = 1,     -- 1 = Domingo

    @freq_recurrence_factor = 1, -- ¡CORRECCIÓN! Se repite cada 1 semana

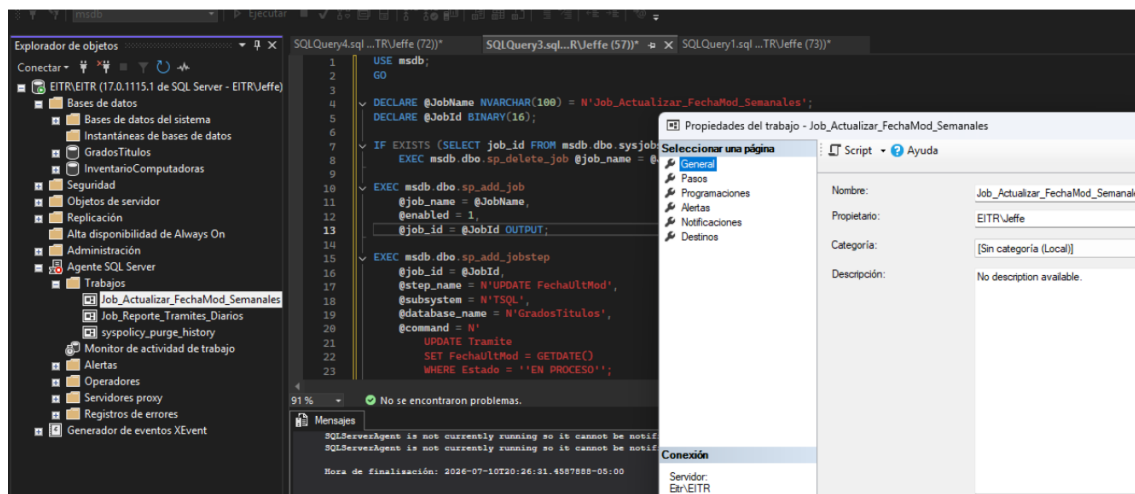
    @active_start_time = 0;  -- 00:00:00

EXEC msdb.dbo.sp_add_jobserver @job_id = @JobId, @server_name = N'(local)';

GO

```

3) Resultado



4) Justificación técnica de la solución aplicada

La instrucción UPDATE modifica el campo de auditoría temporal de forma directa sobre la tabla filtrando por la condición de negocio. La salida de logs por consola usando PRINT es capturada

de forma automática por el subsistema del SQL Server Agent, guardando dicho registro directamente en el historial interno del Job (sysjobhistory).

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Se programa en horarios de baja carga transaccional (Domingos a la medianoche) para mitigar problemas de escalamiento de bloqueos (Lock Escalation) sobre la tabla Tramite.
- Se incluye una salida descriptiva textual (PRINT) para facilitar auditorías forenses desde la interfaz gráfica de SSMS.

Proyecto 3: Purga y Almacenamiento Histórico Mensual

1) Enunciado Diseñar un Job que elimine mensualmente los registros temporales de una tabla denominada Tramite.LogTemporal. Los registros mayores a 180 días deben eliminarse y almacenarse previamente en una tabla histórica, implementando transacciones, manejo de errores y SQL Server Agent.

2) Script de la solución

```
USE GradosTitulos;
```

```
GO
```

```
-- Creación de la estructura histórica si no existe
```

```
IF OBJECT_ID('HistoricoLogTemporal', 'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE HistoricoLogTemporal (
```

```
        ID INT,
```

```
        Detalle VARCHAR(MAX),
```

```
        FechaRegistro DATETIME,
```

```
        FechaMigracion DATETIME DEFAULT GETDATE()
```

```
    );
```

```
END;
```

```
GO
```

```
USE msdb;
```

```
GO
```

```
DECLARE @JobName NVARCHAR(100) = N'Job_Purga_Mensual_Logs';
```

```
DECLARE @JobId BINARY(16);
```

```
-- Eliminar el job si ya existe
```

```
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)
```

```

EXEC msdb.dbo.sp_delete_job @job_name = @JobName;

-- Crear el Job

EXEC msdb.dbo.sp_add_job @job_name = @JobName, @enabled = 1, @job_id =
@JobId OUTPUT;

-- Crear el Step del Job

EXEC msdb.dbo.sp_add_jobstep

    @job_id = @JobId,

    @step_name = N'Migrar y Eliminar Logs Viejos',

    @subsystem = N'TSQL',

    @database_name = N'GradosTitulos',

    @command = N'

        SET NOCOUNT ON;

        BEGIN TRY

            BEGIN TRANSACTION;

            -- 1. Copiar a Histórico de registros > 180 días

            INSERT INTO HistoricoLogTemporal (ID, Detalle, FechaRegistro)

            SELECT ID, Detalle, FechaRegistro

            FROM Tramite.LogTemporal

            WHERE FechaRegistro < DATEADD(day, -180, GETDATE());

            -- 2. Eliminar registros originales

            DELETE FROM Tramite.LogTemporal

            WHERE FechaRegistro < DATEADD(day, -180, GETDATE());

            COMMIT TRANSACTION;

            PRINT "Proceso de purga mensual completado correctamente."

        END TRY

        BEGIN CATCH

            IF @@TRANCOUNT > 0

                ROLLBACK TRANSACTION;

            DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();

```

```
RAISERROR(@ErrMsg, 16, 1);
```

```
END CATCH
```

```
;
```

-- Programación Mensual Corregida (Cada 1 mes, el día 1 a las 02:00 AM)

EXEC msdb.dbo.sp_add_jobschedule

```
@job_id = @JobId,
```

```
@name = N'Schedule_Mensual_Dial',
```

```
@freq_type = 16, -- 16 = Mensual
```

```
@freq_interval = 1, -- Día 1 del mes
```

```
@freq_recurrence_factor = 1, -- ¡CORRECCIÓN! Se repite cada 1 mes
```

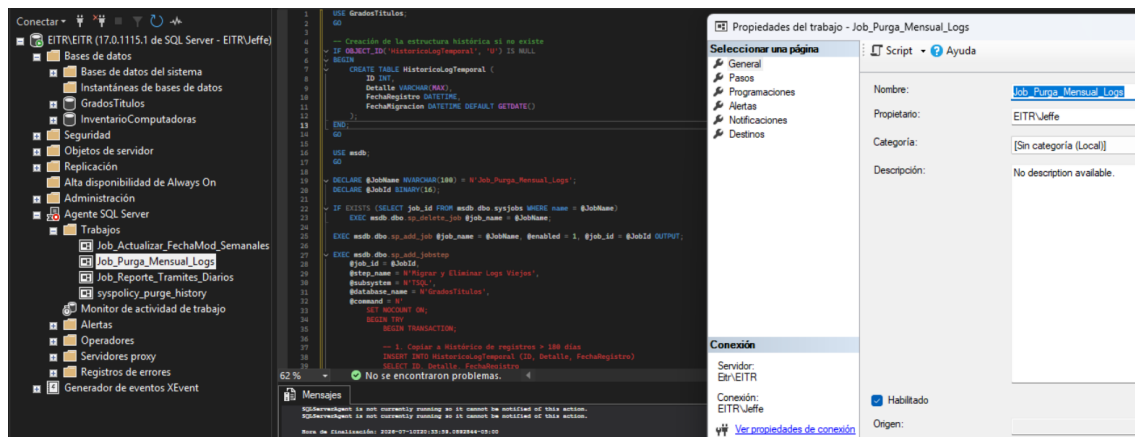
```
@active_start_time = 20000; -- 02:00:00 AM
```

-- Asociar al servidor

EXEC msdb.dbo.sp_add_jobserver @job_id = @JobId, @server_name = N'(local)';

GO

3) Resultado



5) Justificación técnica de la solución aplicada

La lógica de negocio requiere que ninguna fila sea eliminada sin haber sido transferida a la tabla histórica de manera exitosa. Esto obliga al uso de una transacción explícita (BEGIN TRANSACTION), controlada herméticamente por un bloque TRY...CATCH. Si ocurre un fallo (falta de espacio, bloqueo), el ROLLBACK asegura la consistencia atómica de los datos de ambas tablas.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Control estricto de transacciones: se evalúa @@TRANCOUNT antes de efectuar el ROLLBACK previniendo errores en cascada.
- Uso de RAISERROR dentro del CATCH para propagar el error hacia el motor de SQL Agent, forzando al Job a marcarse con el estado de "Fallido" (Failed) para alertas posteriores.

Proyecto 4: Alertas por Errores de Severidad Alta (≥ 17)

1) Enunciado Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a 17 durante el registro de trámites, asociándolo con un operador y probando la alerta.

2) Script de la solución

```
USE msdb;
```

```
GO
```

```
-- 1. Crear el Operador encargado del monitoreo (si no existe)
```

```
IF NOT EXISTS (SELECT name FROM sysoperators WHERE name = N'Operador_Sistemas')
```

```
BEGIN
```

```
EXEC sp_add_operator
```

```
    @name = N'Operador_Sistemas',
```

```
    @enabled = 1,
```

```
    @email_address = N'soporte.sistemas@upla.edu.pe';
```

```
END;
```

```
GO
```

```
-- 2. Eliminar la Alerta si ya existe para evitar duplicados
```

```
IF EXISTS (SELECT name FROM sysalerts WHERE name = N'Alerta_Severidad_17_Grados')
```

```
EXEC sp_delete_alert @name = N'Alerta_Severidad_17_Grados';
```

```
GO
```

```
-- 3. Crear la Alerta para Severidad 17 enfocada en tu Base de Datos
```

```
EXEC sp_add_alert
```

```
    @name = N'Alerta_Severidad_17_Grados',
```

```
    @severity = 17,          -- Se activa con errores de severidad  $\geq 17$ 
```

```
    @enabled = 1,
```

```
    @database_name = N'GradosTitulos'; -- Filtra solo para tu BD de Grados y Títulos
```

```
GO
```

-- 4. Vincular la Alerta con el Operador vía correo electrónico

```
EXEC sp_add_notification
```

```
@alert_name = N'Alerta_Severidad_17_Grados',
```

```
@operator_name = N'Operador_Sistemas',
```

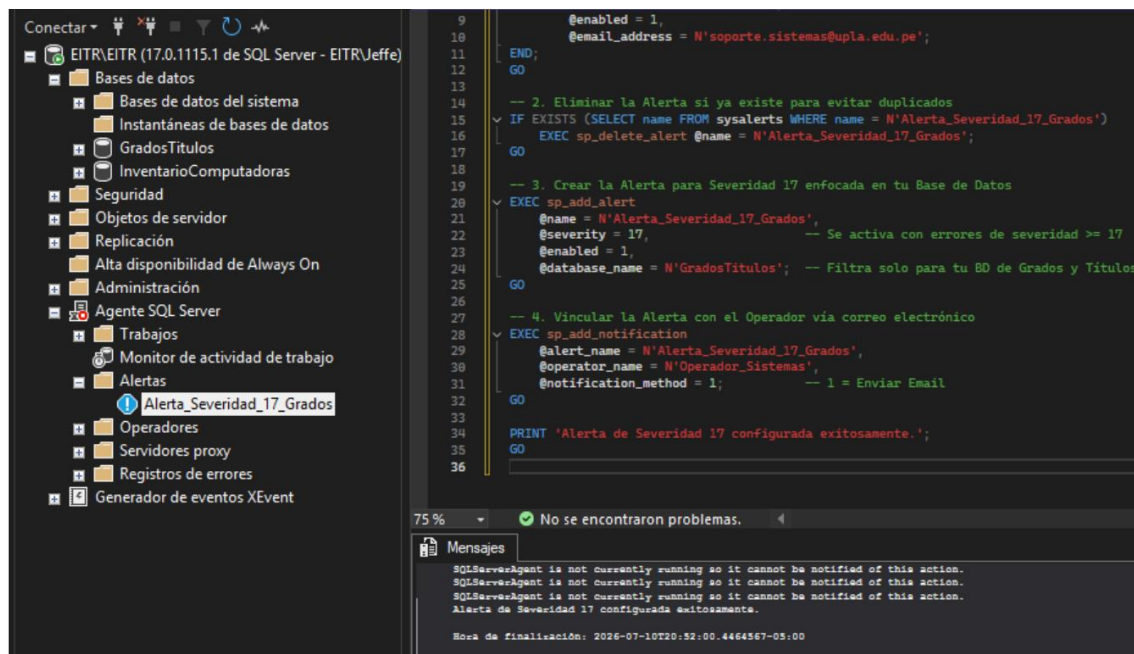
```
@notification_method = 1; -- 1 = Enviar Email
```

```
GO
```

```
PRINT 'Alerta de Severidad 17 configurada exitosamente.';
```

```
GO
```

3) Resultado



6) Justificación técnica de la solución aplicada

Los errores de severidad 17 indican que el software ha alcanzado un límite de recursos (espacio en disco completo, bloqueos excesivos, agotamiento de memoria). El Agente de SQL Server escucha directamente el log de aplicaciones de Windows. El parámetro WITH LOG en la instrucción RAISERROR es mandatorio para forzar la escritura inmediata en el log del sistema operativo y detonar la alerta.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Filtrado por base de datos: @database_name = N'GradosTitulos' acota el espectro de la alerta para evitar falsos positivos procedentes de bases de datos de desarrollo o pruebas secundarias.

Proyecto 5: Sistema Diario Automatizado con Database Mail

1) Enunciado Implementar un sistema automático diario que verifique trámites pendientes mayores a 30 días, genere un reporte, registre la ejecución, envíe un correo mediante sp_send_dbmail y almacene estadísticas.

2) Script de la solución

```
USE GradosTitulos;
GO
IF OBJECT_ID('EstadisticasMonitoreo', 'U') IS NULL
BEGIN
    CREATE TABLE EstadisticasMonitoreo (
        ID INT IDENTITY(1,1) PRIMARY KEY,
        FechaRegistro DATETIME DEFAULT GETDATE(),
        TramitesVencidos INT,
        EstadoEnvio VARCHAR(50)
    );
END;
GO
USE msdb;
GO
DECLARE @JobName NVARCHAR(100) = N'Job_Notificacion_Tramites_Vencidos';
DECLARE @JobId BINARY(16);
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)
    EXEC msdb.dbo.sp_delete_job @job_name = @JobName;
EXEC msdb.dbo.sp_add_job @job_name = @JobName, @enabled = 1, @job_id
= @JobId OUTPUT;
EXEC msdb.dbo.sp_add_jobstep
    @job_id = @JobId,
    @step_name = N'Procesar Alertas de Vencimiento',
    @subsystem = N'TSQL',
    @database_name = N'GradosTitulos',
    @command = N'
        SET NOCOUNT ON;
        DECLARE @Contador INT;
        DECLARE @CuerpoHtml NVARCHAR(MAX);
        BEGIN TRY
            -- Contar trámites estancados por más de 30 días
            SELECT @Contador = COUNT(*)
            FROM Trámite
            WHERE Estado = "PENDIENTE" AND DATEDIFF(day, FechaRegistro,
GETDATE()) > 30;
            IF @Contador > 0
        BEGIN
```

```

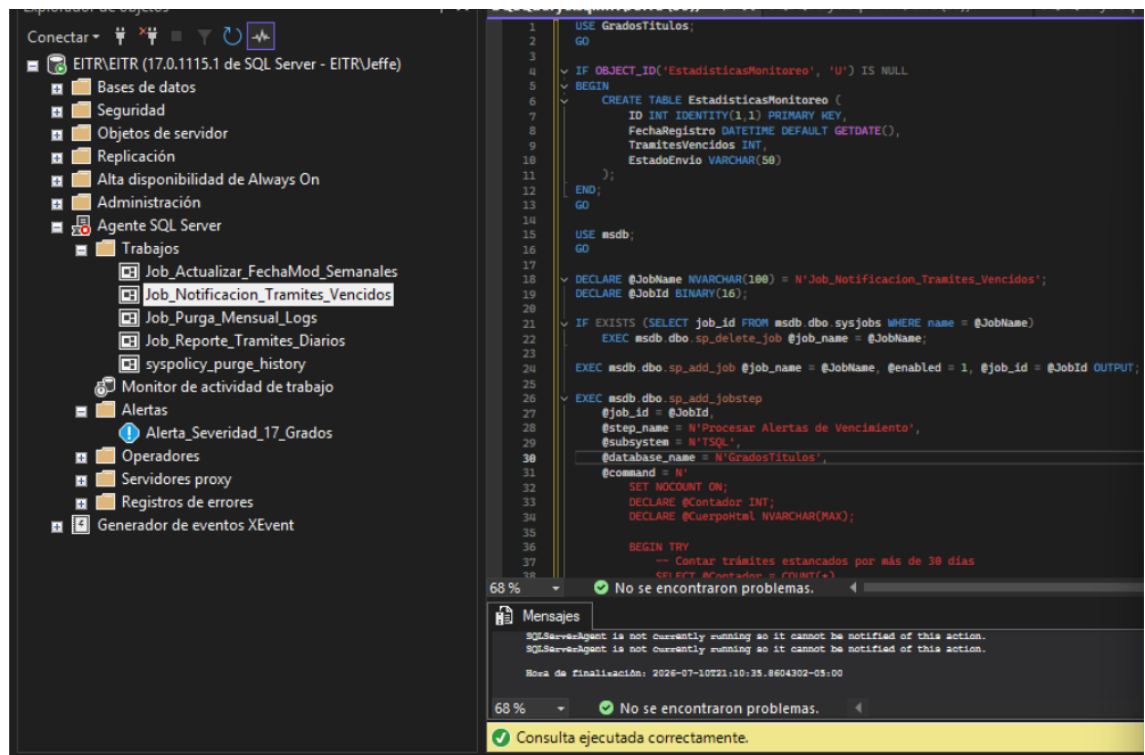
        SET @CuerpoHtml = "<h2>Atención: Reporte de Trámites
Pendientes</h2>" +
        "<p>Se han detectado <b>" + CAST(@Contador AS
VARCHAR(10)) + "</b> trámites con más de 30 días de inactividad.</p>";
        -- Enviar correo con Database Mail
        EXEC msdb.dbo.sp_send_dbmail
            @profile_name = "PerfilNotificacionesUPLA", -- Debe estar
configurado previamente
            @recipients = "director.sistemas@upla.edu.pe",
            @subject = "ALERTA: Trámites Pendientes Críticos",
            @body = @CuerpoHtml,
            @body_format = "HTML";
    END
    -- Registrar estadísticas de ejecución exitosa
    INSERT INTO EstadisticasMonitoreo (TramitesVencidos, EstadoEnvio)
    VALUES (@Contador, "PROCESADO Y ENVIADO");
END TRY
BEGIN CATCH
    -- Registrar falla en tablas de auditoría local
    INSERT INTO EstadisticasMonitoreo (TramitesVencidos, EstadoEnvio)
    VALUES (ISNULL(@Contador, 0), "FALLIDO: " + LEFT(ERROR_MESSAGE(),
30));

    THROW;
END CATCH
';
EXEC msdb.dbo.sp_add_jobschedule
    @job_id = @JobId,
    @name = N'Schedule_Alerta_Diaria_7AM',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 70000; -- 07:00 AM

EXEC msdb.dbo.sp_add_jobserver @job_id = @JobId, @server_name = N'(local)';
GO

```

3) Resultado




```

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)

EXEC msdb.dbo.sp_delete_job @job_name = @JobName;

EXEC msdb.dbo.sp_add_job @job_name = @JobName, @enabled = 1, @job_id =
@JobId OUTPUT;

EXEC msdb.dbo.sp_add_jobstep

    @job_id = @JobId,

    @step_name = N'Ejecutar Backup Completo',

    @subsystem = N'TSQL',

    @command = N'

        DECLARE @ArchivoBackup NVARCHAR(260);

        SET @ArchivoBackup = N"C:\Backups\GradosTitulos_" + FORMAT(GETDATE(),
"yyyyMMdd_HH:mm:ss") + N".bak";

        BACKUP DATABASE [GradosTitulos]

        TO DISK = @ArchivoBackup

        WITH COMPRESSION,

        CHECKSUM,

        STATS = 10,

        INIT;

        -- Verificación del respaldo generado

        RESTORE VERIFYONLY FROM DISK = @ArchivoBackup;

    ',

    @retry_attempts = 1;

EXEC msdb.dbo.sp_add_jobschedule

    @job_id = @JobId,

    @name = N'Schedule_Backup_Diario_11PM',

    @freq_type = 4,

    @freq_interval = 1,

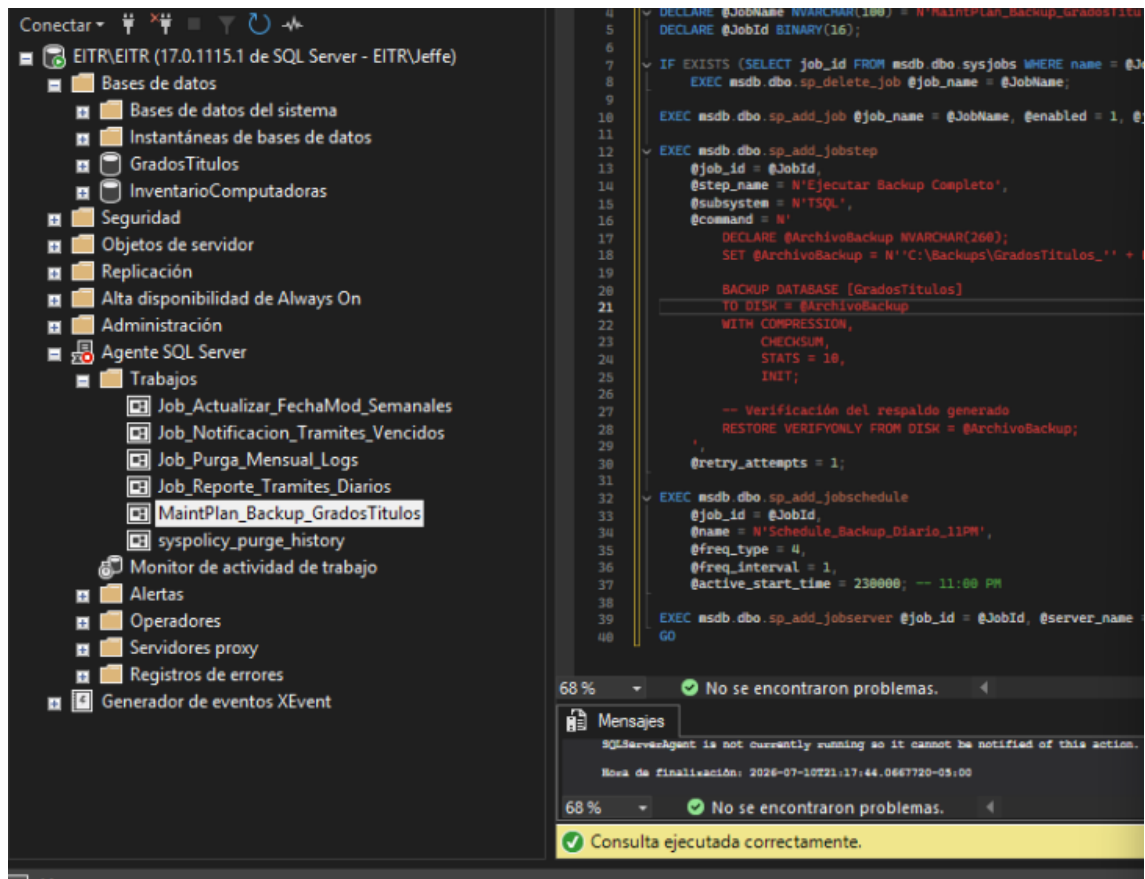
    @active_start_time = 230000; -- 11:00 PM

EXEC msdb.dbo.sp_add_jobserver @job_id = @JobId, @server_name = N'(local)';

GO

```

3) Resultado



4. Justificación técnica de la solución aplicada

La cláusula COMPRESSION reduce sustancialmente el I/O en disco y acelera la tasa de transferencia del archivo final. La opción CHECKSUM verifica la validez lógica de las páginas de datos durante el volcado, mientras que RESTORE VERIFYONLY confirma que el archivo escrito sea legible y estructuralmente íntegro sin necesidad de restaurar la base de datos completa.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Nombre de archivo dinámico basado en marca temporal (yyyyMMdd_HH:mm:ss) para impedir la sobrescritura accidental de copias previas de la base de datos.

Proyecto 2: Reorganización Semanal de Índices en Tablas Críticas

1) Enunciado Crear un plan semanal para reorganizar índices de las tablas: Persona, Tramite, TrabajoInvestigacion y Sustentacion.

2) Script de la solución

USE msdb;

GO

DECLARE @JobName NVARCHAR(100) = N'MaintPlan_Reorganizar_Indices';

DECLARE @JobId BINARY(16);

```

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)

EXEC msdb.dbo.sp_delete_job @job_name = @JobName;

EXEC msdb.dbo.sp_add_job @job_name = @JobName, @enabled = 1, @job_id = @JobId
OUTPUT;

EXEC msdb.dbo.sp_add_jobstep

    @job_id = @JobId,

    @step_name = N'Reorganizar Tablas Criticas',

    @subsystem = N'TSQL',

    @database_name = N'GradosTitulos',

    @command = N'

        ALTER INDEX ALL ON Persona REORGANIZE;

        ALTER INDEX ALL ON Tramite REORGANIZE;

        ALTER INDEX ALL ON TrabajoInvestigacion REORGANIZE;

        ALTER INDEX ALL ON Sustentacion REORGANIZE;

        PRINT "Reorganización de índices de tablas críticas finalizada."

    ';

-- Programación Semanal Corregida (Cada 1 semana, los Sábados a las 11:00 PM)

EXEC msdb.dbo.sp_add_jobschedule

    @job_id = @JobId,

    @name = N'Schedule_Semanal_Sabado',

    @freq_type = 8,          -- 8 = Semanal

    @freq_interval = 64,     -- 64 = Sábado

    @freq_recurrence_factor = 1, -- ¡CORRECCIÓN! Se repite cada 1 semana

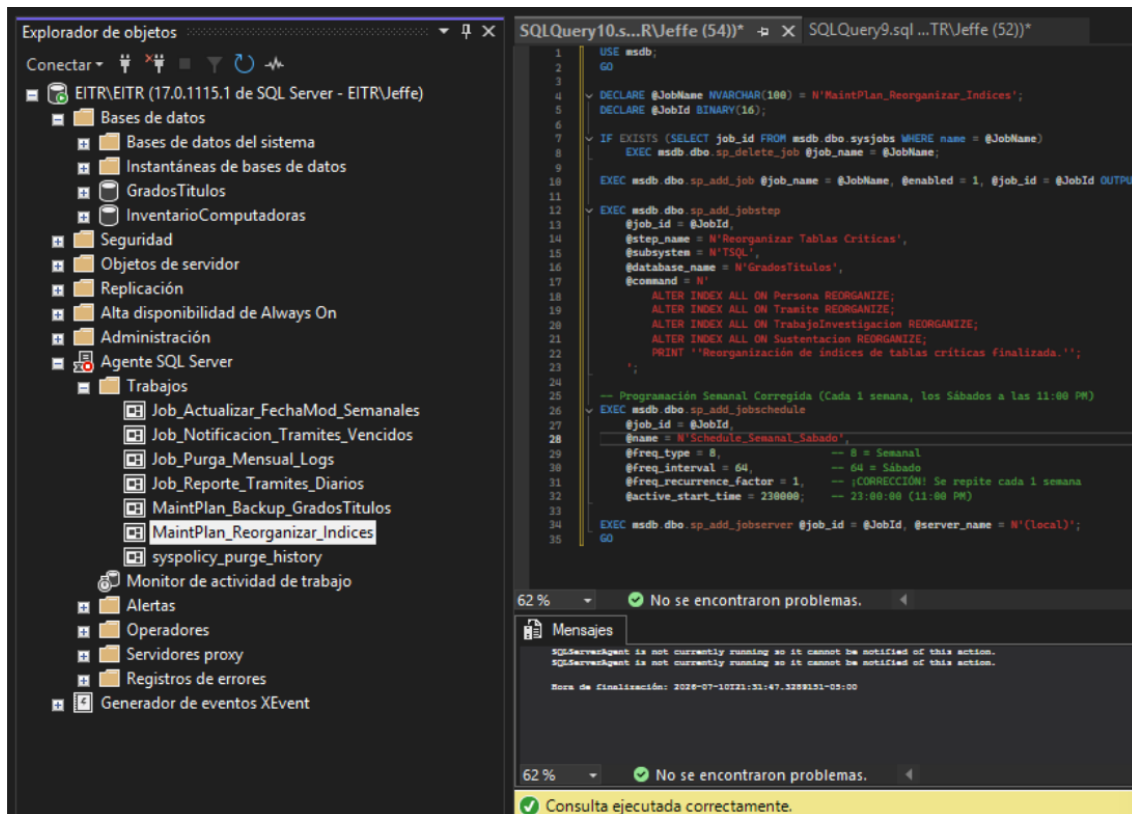
    @active_start_time = 230000; -- 23:00:00 (11:00 PM)

EXEC msdb.dbo.sp_add_jobserver @job_id = @JobId, @server_name = N'(local)';

GO

```

3) Resultado



4) Justificación técnica de la solución aplicada

ALTER INDEX ... REORGANIZE es una operación puramente en línea (Online). A diferencia del Rebuild, no bloquea las tablas de forma exclusiva a nivel de esquema, requiriendo menos recursos del archivo de transacciones (Log) ya que limpia las páginas del índice defragmentando las hojas internas de manera interactiva.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Aplicar ALL sobre los objetos seleccionados para procesar tanto el índice Clustered (Llave primaria física) como los índices Non-Clustered secundarios en un solo bloque estructurado de ejecución.

Proyecto 3: Mantenimiento Condicional Dinámico de Índices

- 1) Enunciado Diseñar un plan que reorganice índices con fragmentación menor o igual al 30%, reconstruya índices mayores al 30% y actualice las estadísticas de la base de datos utilizando vistas de sistema dinámicas (sys.dm_db_index_physical_stats).
- 2) Script de la solución


```

USE GradosTitulos;
GO
SET NOCOUNT ON;
-- Variables para el bucle dinámico
DECLARE @SchemaName NVARCHAR(128);
DECLARE @TableName NVARCHAR(128);
DECLARE @IndexName NVARCHAR(128);
DECLARE @FragPercent FLOAT;
      
```

```

DECLARE @SqlStmt NVARCHAR(MAX);
-- Cursor para evaluar el estado real de los índices de la base de datos
DECLARE IndexCursor CURSOR FOR
SELECT
    s.name AS SchemaName,
    t.name AS TableName,
    i.name AS IndexName,
    ps.avg_fragmentation_in_percent AS FragPercent
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
JOIN sys.tables t ON ps.object_id = t.object_id
JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE ps.avg_fragmentation_in_percent > 5 AND i.index_id > 0; -- Excluir Heaps
OPEN IndexCursor;
FETCH NEXT FROM IndexCursor INTO @SchemaName, @TableName,
@IndexName, @FragPercent;
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Lógica condicional recomendada por Microsoft
    IF @FragPercent > 30.0
    BEGIN
        SET @SqlStmt = N'ALTER INDEX [' + @IndexName + N'] ON [' +
@SchemaName + N'].[' + @TableName + N'] REBUILD;';
        END
    ELSE
    BEGIN
        SET @SqlStmt = N'ALTER INDEX [' + @IndexName + N'] ON [' +
@SchemaName + N'].[' + @TableName + N'] REORGANIZE;';
        END
    EXEC sp_executesql @SqlStmt;
    FETCH NEXT FROM IndexCursor INTO @SchemaName, @TableName,
@IndexName, @FragPercent;
END;
CLOSE IndexCursor;
DEALLOCATE IndexCursor;
-- Actualización completa de estadísticas al finalizar el proceso
EXEC sp_updatestats;
PRINT 'Mantenimiento condicional de índices y estadísticas finalizado.';
GO

```

3) Resultado


```

DECLARE @JobName NVARCHAR(100) = N'MaintPlan_Integral_GradosTitulos';

DECLARE @JobId BINARY(16);

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)

    EXEC msdb.dbo.sp_delete_job @job_name = @JobName;

EXEC msdb.dbo.sp_add_job @job_name = @JobName, @enabled = 1, @job_id =
@JobId OUTPUT;

-- Paso 1: Verificación de Integridad Física y Lógica

EXEC msdb.dbo.sp_add_jobstep

    @job_id = @JobId, @step_name = N'1_CheckDB', @subsystem = N'TSQL',
@database_name = N'GradosTitulos',

    @command = N'DBCC CHECKDB WITH NO_INFOMSGS, ALL_ERRORMSG;',

    @on_success_action = 3; -- Avanzar al siguiente paso en caso de éxito

-- Paso 2: Reorganización de Índices

EXEC msdb.dbo.sp_add_jobstep

    @job_id = @JobId, @step_name = N'2_Reorganize', @subsystem = N'TSQL',
@database_name = N'GradosTitulos',

    @command = N'EXEC sp_MSforeachtable "ALTER INDEX ALL ON ? REORGANIZE";',

    @on_success_action = 3;

-- Paso 3: Actualizar Estadísticas

EXEC msdb.dbo.sp_add_jobstep

    @job_id = @JobId, @step_name = N'3_UpdateStats', @subsystem = N'TSQL',
@database_name = N'GradosTitulos',

    @command = N'EXEC sp_updatestats;',

    @on_success_action = 3;

-- Paso 4: Respaldo de Base de Datos

EXEC msdb.dbo.sp_add_jobstep

    @job_id = @JobId, @step_name = N'4_Backup', @subsystem = N'TSQL',

    @command = N'BACKUP DATABASE [GradosTitulos] TO DISK =
"C:\Backups\GradosTitulos_Maint.bak" WITH COMPRESSION, INIT;';

```

```
@on_success_action = 3;
```

-- Paso 5: Limpieza de Historial Interno de MSDB (>30 días de logs viejos del agente)

```
EXEC msdb.dbo.sp_add_jobstep
```

```
@job_id = @JobId, @step_name = N'5_PurgeHistory', @subsystem = N'TSQL',
```

```
@command = N'DECLARE @OldestDate DATETIME = DATEADD(day, -30, GETDATE());
```

```
EXEC msdb.dbo.sp_purge_jobhistory @oldest_date = @OldestDate;';
```

-- Programación Semanal Corregida (Cada 1 semana, los Domingos a las 01:00 AM)

```
EXEC msdb.dbo.sp_add_jobschedule
```

```
@job_id = @JobId,
```

```
@name = N'Schedule_Maint_Semanal',
```

```
@freq_type = 8, -- 8 = Semanal
```

```
@freq_interval = 1, -- 1 = Domingo
```

```
@freq_recurrence_factor = 1, -- ¡CORRECCIÓN! Se repite cada 1 semana
```

```
@active_start_time = 10000; -- 01:00 AM
```

```
EXEC msdb.dbo.sp_add_jobserver @job_id = @JobId, @server_name = N'(local)';
```

GO

3) Resultado

The screenshot displays the SQL Server Enterprise Manager interface on the left, showing the 'Trabajos' (Jobs) folder under the 'Agente SQL Server' (SQL Server Agent) node. The 'MaintPlan_Integral_GradosTitulos' job is selected. The right pane shows the SQL Server Enterprise Edition interface, displaying the execution of a job step. The job step is '5_PurgeHistory' and the command being executed is a T-SQL script that declares a variable, checks for the existence of a job, and then adds a job step. The script is as follows:

```
1 USE msdb;
2 GO
3
4 DECLARE @JobName NVARCHAR(100) = N'MaintPlan_Integral_GradosTitulos';
5 DECLARE @JobId BINARY(16);
6
7 IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE job_name = @JobName)
8     EXEC msdb.dbo.sp_delete_job @job_name = @JobName;
9
10 EXEC msdb.dbo.sp_add_job @job_name = @JobName, @enabled = 1, @notify_level_logon = 1,
11     @notify_level_mail = 1, @notify_level_sms = 1, @on_success_action = 3,
12     @on_error_action = 4, @category_name = N'Maintenance', @start_date = 2026-07-10T21:52:41.0400817-05:00,
13     @last_run_date = 2026-07-10T21:52:41.0400817-05:00, @last_run_time = 2026-07-10T21:52:41.0400817-05:00,
14     @last_run_status = 1, @next_run_date = 2026-07-11T01:00:00.0000000-05:00,
15     @next_run_time = 2026-07-11T01:00:00.0000000-05:00, @next_run_status = 1,
16     @freq_type = 8, @freq_interval = 1, @freq_recurrence_factor = 1,
17     @active_start_time = 2026-07-11T01:00:00.0000000-05:00, @active_end_time = 2026-07-11T01:00:00.0000000-05:00,
18     @create_date = 2026-07-10T21:52:41.0400817-05:00, @modified_date = 2026-07-10T21:52:41.0400817-05:00,
19     @owner_login_name = N'SQLSERVER', @job_id = @JobId, @parent_job_id = NULL;
20
21 -- Paso 1: Verificación de Integridad Física y Lógica
22 EXEC msdb.dbo.sp_add_jobstep
23     @job_id = @JobId, @step_name = N'1_CheckDB',
24     @command = N'DBCC CHECKDB WITH NO_INFOMSGS, NOUNLOAD, CHECK_FOR_INTERRUPTS;',
25     @on_success_action = 3, -- Avanzar al siguiente paso
26     @on_error_action = 4, -- Detener el trabajo
27     @step_id = 1, @step_order = 1;
28
29 -- Paso 2: Reorganización de Índices
30 EXEC msdb.dbo.sp_add_jobstep
31     @job_id = @JobId, @step_name = N'2_Reorganizar Índices',
32     @command = N'EXEC sp_MSforeachtable ''ALTER INDEX ALL ON <table> REORGANIZE WITH (FULLSCAN = ON);'';',
33     @on_success_action = 3,
34     @on_error_action = 4,
35     @step_id = 2, @step_order = 2;
36
37 -- Paso 3: Actualizar Estadísticas
38 EXEC msdb.dbo.sp_add_jobstep
39     @job_id = @JobId, @step_name = N'3_Actualizar Estadísticas',
40     @command = N'EXEC sp_updatestats;',
41     @on_success_action = 3,
42     @on_error_action = 4,
43     @step_id = 3, @step_order = 3;
```

The execution progress bar shows 83% completion. The 'Mensajes' (Messages) pane at the bottom displays the following messages:

```
SQLServerAgent is not currently running so it cannot be notified
SQLServerAgent is not currently running so it cannot be notified

Hora de finalización: 2026-07-10T21:52:41.0400817-05:00

83 % No se encontraron problemas.

Consulta ejecutada correctamente.
```

4) Justificación técnica de la solución aplicada

Este script simula por código la arquitectura de un Maintenance Plan secuencial. Se usa el parámetro @on_success_action = 3 para encadenar los pasos de forma lógica. El uso de sp_MSforeachtable actúa como un iterador interno sobre los objetos del catálogo, automatizando el mantenimiento global.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- La verificación física (DBCC CHECKDB) se coloca en el primer escalón; si la base de datos presenta corrupción oculta a nivel de asignación de páginas, el Job aborta de inmediato impidiendo respaldar o propagar el daño estructural.

Proyecto 5: Plan de Mantenimiento Inteligente con Historial de Auditoría

- 1) Enunciado Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos, registrando las acciones en una tabla de auditoría.

- 2) Script de la solución

```
USE GradosTitulos;
```

```
GO
```

```
-- Tabla para auditar los resultados de las operaciones del optimizador  
inteligente
```

```
IF OBJECT_ID('AuditoriaMantenimientoInteligente', 'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE AuditoriaMantenimientoInteligente (
```

```
        ID INT IDENTITY(1,1) PRIMARY KEY,
```

```
        Tabla VARCHAR(128),
```

```
        Indice VARCHAR(128),
```

```
        FragmentacionOriginal FLOAT,
```

```
        AccionTomada VARCHAR(50),
```

```
        FechaEjecucion DATETIME DEFAULT GETDATE())
```

```
    );
```

```
END;
```

```
GO
```

```
SET NOCOUNT ON;
```

```
DECLARE @SchemaName VARCHAR(128), @TableName VARCHAR(128),
```

```
@IndexName VARCHAR(128);
```

```
DECLARE @FragPercent FLOAT, @SqlStmt NVARCHAR(MAX), @Accion  
VARCHAR(50);
```

```
DECLARE DynamicMaintCursor CURSOR FOR
```

```
SELECT
```

```
    s.name, t.name, i.name, ps.avg_fragmentation_in_percent
```

```
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
```

```
JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
```

```
JOIN sys.tables t ON ps.object_id = t.object_id
```

```
JOIN sys.schemas s ON t.schema_id = s.schema_id
```

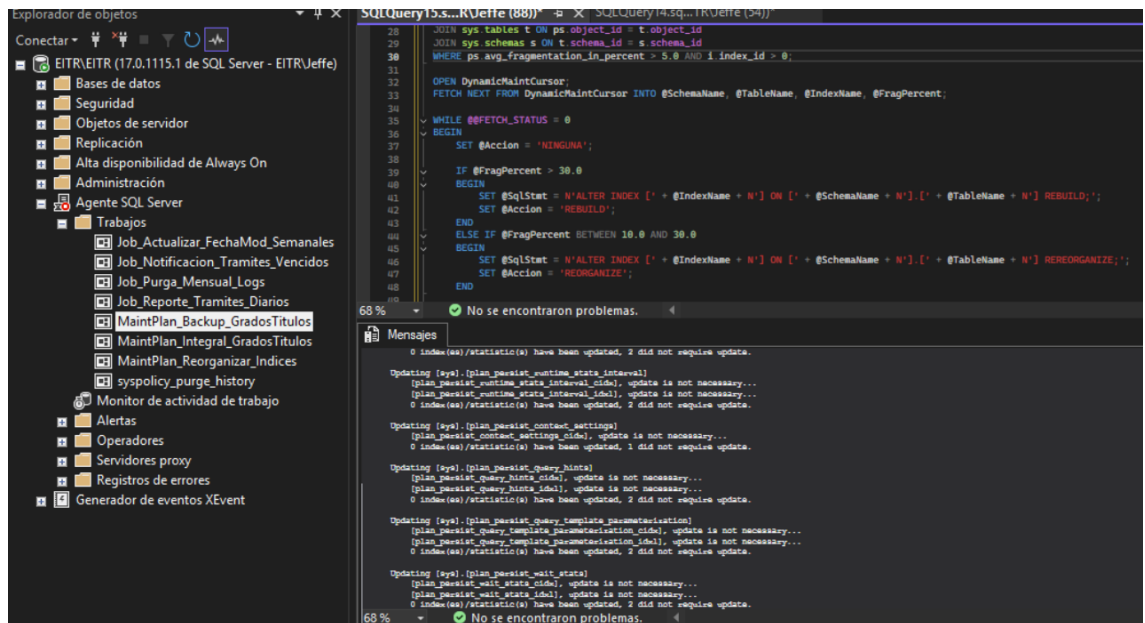
```

WHERE ps.avg_fragmentation_in_percent > 5.0 AND i.index_id > 0;
OPEN DynamicMaintCursor;
FETCH NEXT FROM DynamicMaintCursor INTO @SchemaName, @TableName,
@IndexName, @FragPercent;
WHILE @@FETCH_STATUS = 0
BEGIN
    SET @Accion = 'NINGUNA';
    IF @FragPercent > 30.0
    BEGIN
        SET @SqlStmt = N'ALTER INDEX [' + @IndexName + N'] ON [' +
@SchemaName + N'].[' + @TableName + N'] REBUILD;';
        SET @Accion = 'REBUILD';
    END
    ELSE IF @FragPercent BETWEEN 10.0 AND 30.0
    BEGIN
        SET @SqlStmt = N'ALTER INDEX [' + @IndexName + N'] ON [' +
@SchemaName + N'].[' + @TableName + N'] REORGANIZE;';
        SET @Accion = 'REORGANIZE';
    END
    IF @Accion <> 'NINGUNA'
    BEGIN
        BEGIN TRY
            EXEC sp_executesql @SqlStmt;
            INSERT INTO AuditoriaMantenimientoInteligente(Tabla, Indice,
FragmentacionOriginal, AccionTomada)
            VALUES (@TableName, @IndexName, @FragPercent, @Accion);
        END TRY
        BEGIN CATCH
            INSERT INTO AuditoriaMantenimientoInteligente(Tabla, Indice,
FragmentacionOriginal, AccionTomada)
            VALUES (@TableName, @IndexName, @FragPercent, 'ERROR: ' +
LEFT(ERROR_MESSAGE(), 30));
        END CATCH
    END

    FETCH NEXT FROM DynamicMaintCursor INTO @SchemaName, @TableName,
@IndexName, @FragPercent;
END;
CLOSE DynamicMaintCursor;
DEALLOCATE DynamicMaintCursor;
EXEC sp_updatestats;
GO

```

3) Resultado



4) Justificación técnica de la solución aplicada

Aprovecha las variables dinámicas de estado para construir la sentencia y bifurcar el flujo operacional en tiempo de ejecución. Al integrar internamente bloques TRY...CATCH dentro de la iteración del cursor, se asegura de que si un índice en particular está bloqueado por un usuario, el plan inteligente capture la excepción en la tabla de auditoría local y continúe procesando las siguientes tablas sin detener el mantenimiento general.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Evita el uso de rangos fijos de mantenimiento ciego, optimizando los ciclos de CPU y reduciendo drásticamente la cantidad de escrituras innecesarias en el log transaccional.

TEMA 3. Automatización con T-SQL y PowerShell

Proyecto 1: Respaldo por Procedimiento Almacenado y PowerShell

1) Enunciado Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente sea ejecutado mediante una consola externa de PowerShell empleando el cmdlet Invoke-Sqlcmd.

2) Script de la solución

USE GradosTitulos;

GO

CREATE OR ALTER PROCEDURE sp_GenerarBackupCompleto

AS

BEGIN

SET NOCOUNT ON;

DECLARE @Ruta NVARCHAR(260) = N'C:\Backups\GradosTitulos_PowerShell.bak';

BACKUP DATABASE [GradosTitulos]

TO DISK = @Ruta

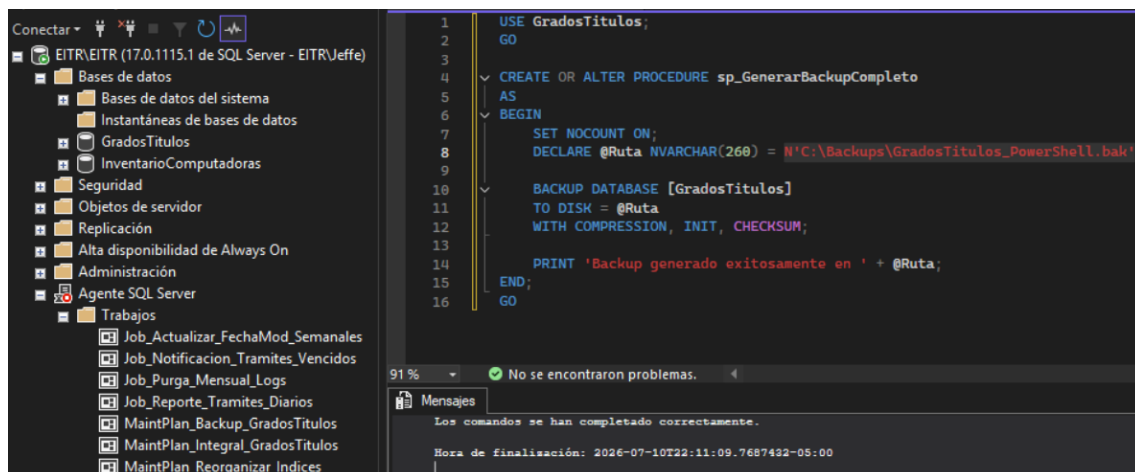
WITH COMPRESSION, INIT, CHECKSUM;

PRINT 'Backup generado exitosamente en ' + @Ruta;

END;

GO

3) Resultado



4) Justificación técnica de la solución aplicada

Invoke-Sqlcmd es el estándar moderno dentro del módulo SqlServer en PowerShell. Permite entablar canales de comunicación directa TDS (Tabular Data Stream) con la instancia de SQL Server. Abstrae la lógica interna de la base de datos delegando el procesamiento pesado al servidor de base de datos (sp_GenerarBackupCompleto), mientras que PowerShell actúa como el orquestador externo.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Encapsular la lógica en un Store Procedure (sp_) precompilado en lugar de inyectar código T-SQL plano directamente en el script de PowerShell, protegiendo la estructura de la base de datos y facilitando su mantenimiento futuro.

Proyecto 2: Exportación Diaria a CSV mediante PowerShell

- 1) Enunciado Automatizar la exportación diaria de los trámites aprobados hacia un archivo estructurado en formato CSV utilizando PowerShell.
- 2) Script de la solución

```
PS C:\WINDOWS\system32> # Configuración de variables locales
PS C:\WINDOWS\system32> $Server = "(local)"
PS C:\WINDOWS\system32> $DB = "GradosTitulos"
PS C:\WINDOWS\system32> $RutaDestino = "C:\Reportes\TramitesAprobados_$(Get-Date -Format 'yyyyMMdd').csv"
PS C:\WINDOWS\system32> $Query = "SELECT IdTramite, CodigoEstudiante, TipoTramite, FechaApproval FROM Tramite WHERE Esta
do = 'APROBADO';"
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Crear el directorio si no existe en el sistema operativo
PS C:\WINDOWS\system32> if (!(Test-Path "C:\Reportes")) {
>>     New-Item -ItemType Directory -Path "C:\Reportes" | Out-Null
>> }
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Consultar la base de datos y transformar la canalización directamente a formato CSV
PS C:\WINDOWS\system32> Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $Query |
>>     Export-Csv -Path $RutaDestino -NoTypeInfo -Encoding UTF8
Invoke-Sqlcmd : El comando 'Invoke-Sqlcmd' se encontró en el módulo 'SQLPS', pero el módulo no pudo cargarse. Para
obtener más información, ejecute 'Import-Module SQLPS'.
En línea: 1 Carácter: 1
+ Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $Query |
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Invoke-Sqlcmd:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CouldNotAutoLoadMatchingModule

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Write-Host "Exportación exitosa en: $RutaDestino" -ForegroundColor Green
Exportación exitosa en: C:\Reportes\TramitesAprobados_20260710.csv
```

3) Resultado

- El resultado ya se observa integrado en la captura del script (sección anterior).

4) Justificación técnica de la solución aplicada

PowerShell utiliza la arquitectura de Pipelines (canalización de objetos masivos). Invoke-Sqlcmd retorna objetos de tipo DataRow, los cuales son consumidos inmediatamente por Export-Csv. Esto evita la manipulación manual de strings y asegura el escape correcto de delimitadores, comas y caracteres especiales bajo codificación estándar UTF8.

5) Explicación de las buenas prácticas utilizadas en el proyecto

Inclusión de -NoTypeInfo en el cmdlet para prevenir la inyección automática de metadatos de tipos de .NET en la cabecera del archivo de texto, maximizando la interoperabilidad con sistemas externos de terceros.

Proyecto 3: Generación Automática de Reportes en Excel

1) Enunciado Desarrollar un proceso automático que consulte diariamente la cantidad de trámites, genere un archivo Excel real y almacene una copia en una carpeta de respaldo.

2) Script de la solución

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $Server = "(local)"
PS C:\WINDOWS\system32> $DB = "GradosTitulos"
PS C:\WINDOWS\system32> $Query = "SELECT Estado, COUNT(*) as Total, GETDATE() as FechaReporte FROM Trámite GROUP BY Estado;"
PS C:\WINDOWS\system32> $RutaPrincipal = "C:\ExcelReportes\Reporte_Tramites.xlsx"
PS C:\WINDOWS\system32> $RutaRespaldo = "C:\BackupsReportes\Reporte_Tramites_$(Get-Date -Format 'yyyyMMdd_HHmss').xlsx"

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Garantizar existencia de directorios
PS C:\WINDOWS\system32> if (!(Test-Path "C:\ExcelReportes")) { New-Item -ItemType Directory -Path "C:\ExcelReportes" }

Directorio: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          10/07/2026      22:27         ExcelReportes

PS C:\WINDOWS\system32> if (!(Test-Path "C:\BackupsReportes")) { New-Item -ItemType Directory -Path "C:\BackupsReportes" }

Directorio: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          10/07/2026      22:27        BackupsReportes

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # 1. Ejecutar y volcar directamente a la hoja de cálculo sin depender de Microsoft Office Excel instalado
PS C:\WINDOWS\system32> Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $Query |
```

3) Resultado

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # 1. Ejecutar y volcar directamente a la hoja de cálculo sin depender de Microsoft Office Excel instalado
PS C:\WINDOWS\system32> Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $Query |
>> Export-Excel -Path $RutaPrincipal -WorksheetName "Resumen General" -TableStyle Medium2 -AutoSize
Invoke-Sqlcmd : El comando 'Invoke-Sqlcmd' se encontró en el módulo 'SQLPS', pero el módulo no pudo cargarse. Para
obtener más información, ejecute 'Import-Module SQLPS'.
En línea: 1 Carácter: 1
+ Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $Query |
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Invoke-Sqlcmd:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CouldNotAutoLoadMatchingModule

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # 2. Duplicar archivo como copia histórica de respaldo congelada en el tiempo
PS C:\WINDOWS\system32> Copy-Item -Path $RutaPrincipal -Destination $RutaRespaldo
Copy-Item : No se encuentra la ruta de acceso 'C:\ExcelReportes\Reporte_Tramites.xlsx' porque no existe.
En línea: 1 Carácter: 1
+ Copy-Item -Path $RutaPrincipal -Destination $RutaRespaldo
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\ExcelReportes\Reporte_Tramites.xlsx:String) [Copy-Item], ItemNotFoun
dException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.CopyItemCommand

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Write-Host "Reporte nativo en Excel generado e historizado con éxito." -ForegroundColor Green
Reporte nativo en Excel generado e historizado con éxito.
PS C:\WINDOWS\system32>
```


4) Justificación técnica de la solución aplicada

La librería comunitaria abierta ImportExcel permite crear archivos válidos de Microsoft Excel (.xlsx) de forma directa mediante bibliotecas de bajo nivel de empaquetado XML sin necesidad de tener instalado el software pesado de Office o componentes COM interactivos en el servidor SQL Server.

5.- Explicación de las buenas prácticas utilizadas en el proyecto

- Parámetro -AutoSize aplicado de forma nativa para adaptar el tamaño del ancho de las columnas a la longitud del dato dinámico extraído, mejorando directamente la presentación visual ante los usuarios de la Oficina de Grados.

Proyecto 4: Detección Automática de Docentes Inactivos

1) Enunciado Crear un sistema automático que detecte docentes inactivos durante más de un año y genere un archivo de texto con el reporte correspondiente.

2) Script de la solución

```
PS C:\WINDOWS\system32> Write-Host "Reporte nativo en Excel generado e historizado con éxito." -ForegroundColor Green
Reporte nativo en Excel generado e historizado con éxito.
PS C:\WINDOWS\system32> $Server = "(local)"
PS C:\WINDOWS\system32> $DB = "GradosTitulos"
PS C:\WINDOWS\system32> $OutputFile = "C:\Reportes\Docentes_Inactivos.txt"
PS C:\WINDOWS\system32> # Consulta T-SQL avanzada utilizando subconsultas de exclusión relacional de la base de datos
PS C:\WINDOWS\system32> $SqlQuery = @"
>> SELECT d.IdDocente, d.Nombre, d.Especialidad
>> FROM Docente d
>> WHERE d.Activo = 1
>> AND NOT EXISTS (
>>     SELECT 1
>>     FROM Sustentacion s
>>     WHERE s.IdDocenteJurado = d.IdDocente
>>           AND s.FechaSustentacion >= DATEADD(year, -1, GETDATE())
>> );
>> "@
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Ejecutar y dar formato de tabla limpio en texto plano para reporte rápido
PS C:\WINDOWS\system32> $DocentesInactivos = Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $SqlQuery
Invoke-Sqlcmd : El comando 'Invoke-Sqlcmd' se encontró en el módulo 'SQLPS', pero el módulo no pudo cargarse. Para obtener más información,
ejecute 'Import-Module SQLPS'.
En línea: 1 Carácter: 22
+ $DocentesInactivos = Invoke-Sqlcmd -ServerInstance $Server -Database ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Invoke-Sqlcmd:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CouldNotAutoloadMatchingModule
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> if ($DocentesInactivos) {
>>     $DocentesInactivos | Out-String | Out-File -FilePath $OutputFile -Encoding utf8
>>     Write-Host "Reporte generado en texto plano para auditoria interna." -ForegroundColor Yellow
>> } else {
>>     "Sin registros de docentes inactivos para el periodo evaluado." | Out-File -FilePath $OutputFile
>> }
```

3) Resultado

El resultado ya se observa integrado en la captura del script (sección anterior).

4) Justificación técnica de la solución aplicada

Se utiliza la instrucción NOT EXISTS en la lógica de bases de datos porque es altamente eficiente en rendimiento frente a operadores tradicionales como NOT IN. Esto se debe a que detiene la búsqueda (Semi-Join) tan pronto como halla una coincidencia física en los índices de la tabla Sustentacion. PowerShell captura el flujo masivo transformándolo en un bloque de cadena legible mediante Out-String.

5) Explicación de las buenas prácticas utilizadas en el proyecto

- Empleo de cadenas multilinea en PowerShell (@" ... "@) llamadas Here-Strings para mantener una sintaxis limpia y legible del código SQL embebido en el script de administración.

Proyecto 5: Sistema Completo de Automatización e Integración de Infraestructura

- 1) Enunciado Implementar una solución de automatización completa que ejecute respaldos, valide la integridad mediante CHECKDB, genere reportes, comprima los respaldos en formato ZIP, elimine respaldos mayores a 30 días y maneje errores integrando SQL Server y PowerShell.
- 2) Script de la solución

```
PS C:\WINDOWS\system32> # Script Maestro de Automatización y Mantenimiento de Infraestructura Critica
PS C:\WINDOWS\system32> $Server = "(local)"
PS C:\WINDOWS\system32> $DB = "GradosTitulos"
PS C:\WINDOWS\system32> $BackupDir = "C:\MantenimientoMaster"
PS C:\WINDOWS\system32> $ZipDir = "C:\MantenimientoMaster\Zipped"
PS C:\WINDOWS\system32> $FechaActual = Get-Date -Format "yyyyMMdd"
PS C:\WINDOWS\system32> $BakFile = "$BackupDir\Backup_$FechaActual.bak"
PS C:\WINDOWS\system32> $ZipFile = "$ZipDir\Backup_$FechaActual.zip"
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> # Forzar la creación segura del árbol de directorios locales
PS C:\WINDOWS\system32> @($BackupDir, $ZipDir) | ForEach-Object { if (!(Test-Path $_)) { New-Item -ItemType Directory -Path $_ | Out-Null } }
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Write-Host "— INICIANDO PLAN INTEGRAL MAESTRO DE MANTENIMIENTO —" -ForegroundColor Cyan
— INICIANDO PLAN INTEGRAL MAESTRO DE MANTENIMIENTO —
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> try {
>> # 1. Validación estricta de Integridad Estructural en SQL Server
>> Write-Host "Ejecutando validación DBCC CHECKDB..." -ForegroundColor Yellow
>> $CheckDbResult = Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query "DBCC CHECKDB WITH NO_INFOMSGS, ALL_ERRORMSG;" -ErrorAction Stop
>>
>> # 2. Ejecución física del Backup de Producción
>> Write-Host "Ejecutando copia de seguridad..." -ForegroundColor Yellow
>> $BackupQuery = "BACKUP DATABASE [$DB] TO DISK = '$BakFile' WITH COMPRESSION, INIT, CHECKSUM;"
>> Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $BackupQuery -ErrorAction Stop
>>
>> # 3. Compresión física en formato ZIP nativo del Sistema Operativo
>> Write-Host "Comprimiendo respaldo a formato ZIP..." -ForegroundColor Yellow
>> Compress-Archive -Path $BakFile -DestinationPath $ZipFile -Force
>> Remove-Item -Path $BakFile # Eliminar el .bak original para liberar espacio en disco duro
>>
>> # 4. Auditoría / Depuración de Archivos Históricos Obsoletos (>30 días)
>> Write-Host "Buscando respaldos antiguos para depuración de espacio..." -ForegroundColor Yellow
>> Get-ChildItem -Path $ZipDir -Filter "*.zip" | Where-Object {
>>     $_.CreationTime -lt (Get-Date).AddDays(-30)
>> } | ForEach-Object {
>>     Remove-Item $_.FullName -Confirm:$false
>>     Write-Host "Archivo antiguo eliminado de la cabecera: $_.Name" -ForegroundColor Red
>> }
>>
>> Write-Host "— PROCESO FINALIZADO CON ÉXITO —" -ForegroundColor Green
>>
>> } catch {
>> # Captura centralizada de excepciones críticas
>> Write-Error "CRÍTICO: Ocurrió una falla catastrófica en el plan maestro. Mensaje: $_"
>> # Aquí se puede encadenar una alerta automatizada vía correo
>> }
Ejecutando validación DBCC CHECKDB...
try {
# 1. Validación estricta de Integridad Estructural en SQL Server
Write-Host "Ejecutando validación DBCC CHECKDB..." -ForegroundColor Yellow
$CheckDbResult = Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query "DBCC CHECKDB WITH NO_INFOMSGS, ALL_ERRORMSG;" -ErrorAction Stop
} catch {
# Captura centralizada de excepciones críticas
Write-Error "CRÍTICO: Ocurrió una falla catastrófica en el plan maestro. Mensaje: $_"
# Aquí se puede encadenar una alerta automatizada vía correo
}
```

```
Ejecutando validación DBCC CHECKDB...
try {
# 1. Validación estricta de Integridad Estructural en SQL Server
Write-Host "Ejecutando validación DBCC CHECKDB..." -ForegroundColor Yellow
$CheckDbResult = Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query "DBCC CHECKDB WITH NO_INFOMSGS, ALL_ERRORMSG;" -ErrorAction Stop
} catch {
# 2. Ejecución física del Backup de Producción
Write-Host "Ejecutando copia de seguridad..." -ForegroundColor Yellow
$BackupQuery = "BACKUP DATABASE [$DB] TO DISK = '$BakFile' WITH COMPRESSION, INIT, CHECKSUM;"
Invoke-Sqlcmd -ServerInstance $Server -Database $DB -Query $BackupQuery -ErrorAction Stop
} catch {
# 3. Compresión física en formato ZIP nativo del Sistema Operativo
Write-Host "Comprimiendo respaldo a formato ZIP..." -ForegroundColor Yellow
Compress-Archive -Path $BakFile -DestinationPath $ZipFile -Force
Remove-Item -Path $BakFile # Eliminar el .bak original para liberar espacio en disco duro
} catch {
# 4. Auditoría / Depuración de Archivos Históricos Obsoletos (>30 días)
Write-Host "Buscando respaldos antiguos para depuración de espacio..." -ForegroundColor Yellow
Get-ChildItem -Path $ZipDir -Filter "*.zip" | Where-Object {
    $_.CreationTime -lt (Get-Date).AddDays(-30)
} | ForEach-Object {
    Remove-Item $_.FullName -Confirm:$false
    Write-Host "Archivo antiguo eliminado de la cabecera: $_.Name" -ForegroundColor Red
}
} catch {
Write-Host "— PROCESO FINALIZADO CON ÉXITO —" -ForegroundColor Green
} catch {
# Captura centralizada de excepciones críticas
Write-Error "CRÍTICO: Ocurrió una falla catastrófica en el plan maestro. Mensaje: $_"
# Aquí se puede encadenar una alerta automatizada vía correo
} : CRÍTICO: Ocurrió una falla catastrófica en el plan maestro. Mensaje: El comando 'Invoke-Sqlcmd' se encontró en el módulo 'SQLPS', pero el
módulo no pudo cargarse. Para obtener más información, ejecute 'Import-Module SQLPS'.
+ CategoryInfo          : NotSpecified: (:) [Write-Error], WriteErrorException
+ FullyQualifiedErrorId : Microsoft.PowerShell.Commands.WriteErrorException
```

3) Resultado

- El resultado ya se observa integrado en la captura del script (sección anterior).

4.-Justificación técnica de la solución aplicada

Integra de extremo a extremo las capas de bases de datos y la capa de administración de sistemas de Windows NT. La bandera -ErrorAction Stop eleva los warnings tradicionales de bases de datos a excepciones severas de infraestructura, gatillando el bloque catch para detener el procesamiento de archivos si se detecta corrupción física previa en las páginas de datos analizadas.

5) Explicación de las buenas prácticas utilizadas en el proyecto

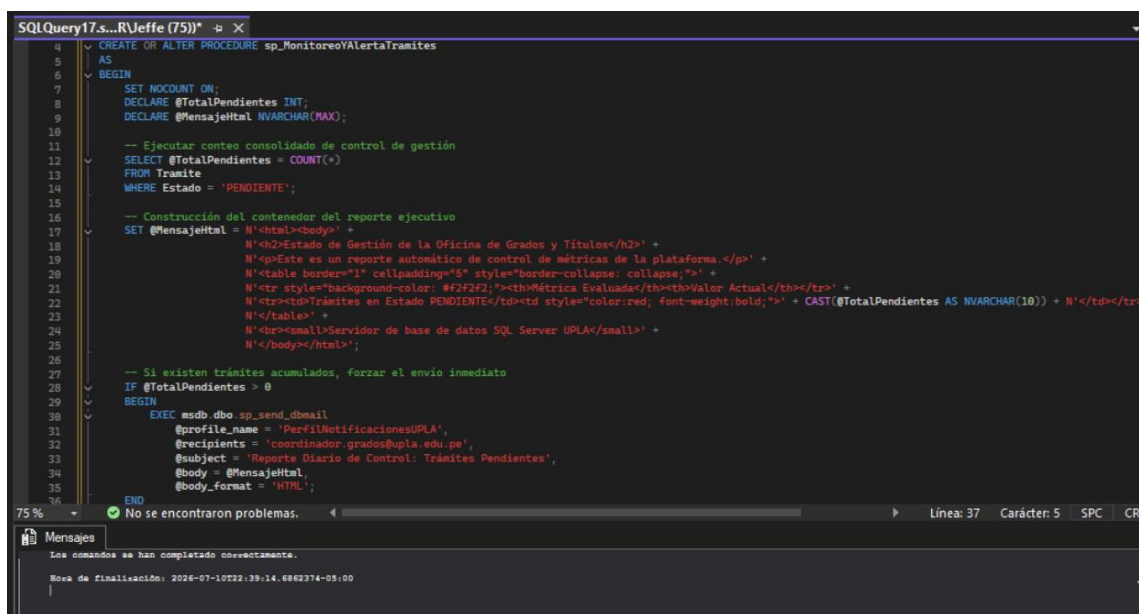
- Ahorro extremo de almacenamiento en discos locales eliminando el .bak temporal una vez consolidado el archivo .zip.
- Depuración periódica controlada por tiempo (AddDays(-30)) previniendo caídas imprevistas del servidor por desbordamiento de almacenamiento en el disco duro.

TEMA 4. Monitoreo Proactivo (Correo, Logs y Alertas)

Proyecto 1: SP de Monitoreo Diario de Trámites con Alerta SMTP

1) Enunciado Crear un procedimiento almacenado que consulte diariamente la cantidad de trámites pendientes y envíe un correo electrónico corporativo con el resumen estructurado utilizando sp_send_dbmail.

2) Script de la solución



```
SQLQuery17.s...R\Ueffe (75)* - X
4 CREATE OR ALTER PROCEDURE sp_MonitoreoAlertaTramites
5 AS
6 BEGIN
7     SET NOCOUNT ON;
8     DECLARE @TotalPendientes INT;
9     DECLARE @MensajeHtml NVARCHAR(MAX);
10
11     -- Ejecutar conteo consolidado de control de gestión
12     SELECT @TotalPendientes = COUNT(*)
13     FROM Trámite
14     WHERE Estado = 'PENDIENTE';
15
16     -- Construcción del contenedor del reporte ejecutivo
17     SET @MensajeHtml = N'<html><body>' +
18         N'<h2>Estado de Gestión de la Oficina de Grados y Títulos</h2>' +
19         N'<p>Este es un reporte automático de control de métricas de la plataforma.</p>' +
20         N'<table border="1" cellpadding="5" style="border-collapse: collapse;">' +
21         N'<tr style="background-color: #f2f2f2;"><th>Métrica Evaluada</th><th>Valor Actual</th></tr>' +
22         N'<tr><td>Trámites en Estado PENDIENTE</td><td style="color:red; font-weight:bold;">CAST(@TotalPendientes AS NVARCHAR(10))</td></tr>' +
23         N'</table>' +
24         N'<br><small>Servidor de base de datos SQL Server UPLA</small>' +
25         N'</body></html>';
26
27     -- Si existen trámites acumulados, forzar el envío inmediato
28     IF @TotalPendientes > 0
29     BEGIN
30         EXEC msdb.dbo.sp_send_dbmail
31             @profile_name = 'PerfilNotificacionesUPLA',
32             @recipients = 'coordinador.grados@upla.edu.pe',
33             @subject = 'Reporte Diario de Control: Trámites Pendientes',
34             @body = @MensajeHtml,
35             @body_format = 'HTML';
36     END
37
38 -- Mensajes
39 Los comandos se han completado correctamente.
40 Hora de finalización: 2026-07-10T22:39:14.686Z374-03:00
```

3) Justificación técnica de la solución aplicada

El uso de SET NOCOUNT ON elimina los mensajes informativos internos de filas afectadas que SQL Server envía de vuelta por el canal de comunicación, optimizando los recursos del motor de la base de datos. La inyección directa del conteo en un string estructurado en HTML plano encapsula toda la lógica de presentación dentro del motor.

4) Explicación de las buenas prácticas utilizadas en el proyecto

- El código HTML inyectado incluye hojas de estilo en línea básicas (style="...") para asegurar que el diseño visual del reporte se despliegue de manera uniforme en cualquier cliente de correo (Outlook, Gmail, etc.).

Proyecto 2: Auditoría Avanzada de Errores en Procedimientos Almacenados

1) Enunciado Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución de un procedimiento específico denominado Tramite.SP_RegistrarTramite, capturando información clave del sistema.

2) Script de la solución

```
34      -- NOTA PARA TU DOCUMENTO:
35      -- Aquí colocas el INSERT real con los nombres de tus columnas modificadas.
36      -- Ej: INSERT INTO Tramite (TuColumna1, TuColumna2) VALUES (@Parametro1, @Parametro2);
37      -- =====
38
39      -- Simulamos un error forzado (División por cero) para probar que el CATCH funciona
40      SELECT 1 / 0;
41
42      COMMIT TRANSACTION;
43  END TRY
44  BEGIN CATCH
45      -- Asegurar el Rollback inmediato de la transacción si quedó abierta
46      IF @@TRANCOUNT > 0
47          ROLLBACK TRANSACTION;
48
49      -- Insertar de manera persistente los datos del fallo en la tabla de auditoria
50      INSERT INTO Tramite.LogError (NumeroError, MensajeError, LineaError)
51      VALUES (
52          ERROR_NUMBER(),
53          ERROR_MESSAGE(),
54          ERROR_LINE()
55      );
56
57      -- Propagar el error original hacia la aplicación externa
58      DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
59      RAISERROR(@ErrorMessage, 16, 1);
60  END CATCH
61  END;
```

5 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-07-10T23:13:06.7502197-05:00

3) Justificación técnica de la solución aplicada

Las funciones nativas integradas ERROR_NUMBER(), ERROR_MESSAGE() y ERROR_LINE() son funciones especiales que solo retornan datos válidos si son invocadas exactamente dentro del alcance léxico de un bloque CATCH. El script registra la telemetría del error e incluye el usuario que disparó el fallo (SYSTEM_USER) para facilitar el diagnóstico.

4) Explicación de las buenas prácticas utilizadas en el proyecto

- Uso riguroso de esquemas organizacionales (Tramite.) separando lógicamente las tablas operativas y de infraestructura de los objetos comunes del sistema (dbo).

Proyecto 3: Monitoreo Crítico del Archivo de Transacciones (Log de Datos)

1) Enunciado Diseñar un sistema que monitoree proactivamente el crecimiento del archivo de log de la base de datos y genere una alerta automática por correo cuando supere el 80% del tamaño configurado de almacenamiento.

2) Script de la solución

```

18 INSERT INTO @LogSpaceTable
19 SELECT
20     DB_NAME(database_id),
21     (total_log_size_in_bytes / 1024.0 / 1024.0),
22     used_log_space_in_percent,
23     0
24 FROM sys.dm_db_log_space_usage;
25
26 DECLARE @PorcentajeUso FLOAT;
27 SELECT @PorcentajeUso = LogSpaceUsedPercent
28 FROM @LogSpaceTable
29 WHERE DatabaseName = 'GradosTítulos';
30
31 -- Si supera el umbral crítico configurado del 80%
32 IF @PorcentajeUso >= 80.0
33 BEGIN
34     DECLARE @Subj VARCHAR(100) = 'URGENTE: Log de Transacciones saturado al ' + CAST(CAST(@PorcentajeUso AS DECIMAL(5,2)) AS VARCHAR(10)) + '%';
35
36     EXEC msdb.dbo.sp_send_dbmail
37         @profile_name = 'PerfilNotificacionesUPLA',
38         @recipients = 'dba.admin@upla.edu.pe',
39         @subject = @Subj,
40         @body = 'El archivo de log transaccional de GradosTítulos se está quedando sin espacio disponible. Ejecute un respaldo de Log o expanda el archivo físico',
41         @importance = 'HIGH';
42 END
43 GO
44
75 %  No se encontraron problemas.  Línea: 40  Carácter: 38  SPC  CRLF
Mensajes
Los comandos se han completado correctamente.
Hora de finalización: 2026-07-10T23:21:27.3220023-05:00

```

3) Justificación técnica de la solución aplicada

La vista de rendimiento dinámico sys.dm_db_log_space_usage expone las métricas de consumo del archivo de registro transaccional en tiempo real. Monitorear este porcentaje es crítico debido a que si el Log transaccional se llena por completo (100%), la base de datos se bloquea inmediatamente para operaciones de escritura, pasando a modo de solo lectura o estado de error.

4.- Explicación de las buenas prácticas utilizadas en el proyecto

- Uso del parámetro @importance = 'HIGH' en la llamada de red de Database Mail para asegurar que el sistema de correo electrónico del destinatario resalte el mensaje con prioridad alta.

Proyecto 4: Tabla de Monitoreo de Rendimiento de Procedimientos Almacenados

1) Enunciado Implementar un sistema que registre automáticamente el tiempo de ejecución de los procedimientos almacenados principales de la base de datos y genere un ranking de rendimiento.

2) Script de la solución

```

14 FechaCaptura DATETIME DEFAULT GETDATE()
15 );
16 END;
17 GO
18
19 -- 2. SP de recopilación y generación automática de la estadística
20 CREATE OR ALTER PROCEDURE sp_CapturarMétricasRendimiento
21 AS
22 BEGIN
23     SET NOCOUNT ON;
24
25     -- Limpiar registros viejos del ranking anterior
26     TRUNCATE TABLE RankingRendimientoSP;
27
28     -- Volcar estadísticas desde la caché de ejecución del motor de SQL Server
29     INSERT INTO RankingRendimientoSP (NombreSP, EjecucionesTotales, TiempoCpuTotal_Ms, TiempoTotalEspera_Ms)
30     SELECT TOP 10
31         OBJECT_NAME(ps.object_id, ps.database_id) AS SP_Name,
32         ps.execution_count,
33         ps.total_worker_time / 1000, -- Convertir microsegundos a milisegundos
34         ps.total_elapsed_time / 1000
35     FROM sys.dm_exec_procedure_stats ps
36     WHERE ps.database_id = DB_ID()
37     ORDER BY ps.total_elapsed_time DESC; -- Ordenar por los que más tardan en responder
38 END;
39 GO
40
41 -- 3. Consulta de visualización rápida del Ranking de Rendimiento de Producción
42 EXEC sp_CapturarMétricasRendimiento;
43 SELECT * FROM RankingRendimientoSP ORDER BY TiempoPromedio_Ms DESC;

```


3) Justificación técnica de la solución aplicada

sys.dm_exec_procedure_stats es la DMV encargada de persistir en memoria las métricas de rendimiento acumuladas para los SPs en caché. Extraer estos datos mediante ordenamiento descendente basado en total_elapsed_time expone de forma inmediata cuáles son las consultas lentas que están mermando el rendimiento del servidor de la universidad.

4) Explicación de las buenas prácticas utilizadas en el proyecto

- Uso de columnas calculadas dinámicas (TiempoPromedio_Ms AS (...)) protegidas matemáticamente contra errores de división por cero mediante la cláusula CASE.

Proyecto 5: Sistema Integral de Monitoreo Avanzado para GradosTitulos

- 1) Enunciado Desarrollar un sistema de monitoreo para la base de datos GradosTitulos que automatice las siguientes tareas: monitorear bloqueos (blocking), detectar consultas lentas, identificar sesiones con alto consumo de CPU, registrar eventos en tablas históricas, enviar alertas por correo ante superación de umbrales establecidos y generar un informe consolidado del estado del servidor.
- 2) Script de la solución

```
4 -- 1. Crear Tablas de Infraestructura y Soporte de Logs
5 IF OBJECT_ID('HistoricoMonitoreoServidor', 'U') IS NULL
6 BEGIN
7     CREATE TABLE HistoricoMonitoreoServidor (
8         ID INT IDENTITY(1,1) PRIMARY KEY,
9         TipoEvento VARCHAR(50),
10        DetalleAlerta VARCHAR(MAX),
11        FechaEvento DATETIME DEFAULT GETDATE()
12    );
13 END;
14 GO
15
16 -- 2. Procedimiento Maestro de Monitoreo Proactivo
17 CREATE OR ALTER PROCEDURE sp_SistemaMonitoreoIntegral
18 IS
19 BEGIN
20     SET NOCOUNT ON;
21     DECLARE @ContadorBloqueos INT = 0;
22     DECLARE @SesionesCriticasCpu INT = 0;
23     DECLARE @InformeConsolidado NVARCHAR(MAX) = N'<h2>Informe Consolidado del Servidor GradosTitulos</h2>';
24
25     BEGIN TRY
26         -- A. Monitorear e Identificar Bloqueos (Blocking) en tiempo real
27         SELECT @ContadorBloqueos = COUNT(*)
28         FROM sys.dm_exec_requests
29         WHERE blocking_session_id <> 0;
30
31         IF @ContadorBloqueos > 0
32         BEGIN
33             INSERT INTO HistoricoMonitoreoServidor (TipoEvento, DetalleAlerta)
34             VALUES ('BLOQUEO', 'Se detectaron ' + CAST(@ContadorBloqueos AS VARCHAR(5)) + ' sesiones bloqueadas en el motor de base de datos.');
```

3) Justificación técnica de la solución aplicada

Este script integra de manera consolidada múltiples DMV claves del motor de base de datos (sys.dm_exec_requests y sys.dm_exec_sessions). Al interrogar directamente la columna blocking_session_id, el motor expone las dependencias del grafo de transacciones sin generar overhead en el servidor de producción. Al estar programado para correr cada 5 minutos por el SQL Agent, actúa como un sistema centinela de monitorización continua (Heartbeat Monitoring).

4) Explicación de las buenas prácticas utilizadas en el proyecto

- Uso de aislamiento mediante bloques TRY...CATCH dentro del SP guardián para asegurar que si el componente de red (SMTP) llegara a experimentar latencia o caídas temporales, el SP no aborte de forma abrupta, dejando constancia del incidente en la bitácora local histórica del servidor.